Phala Network Runtime 구현 분석

1. Runtime 개요

Phala Node Runtime은 **Substrate 기반의 블록체인 런타임**으로, WASM으로 컴파일되어 온체인에서 실행되는 상태 전이 함수입니다. Phala Network의 모든 비즈니스 로직과 거버넌스 규칙을 정의합니다.

2. Runtime 구조 분석

2.1 핵심 팰릿 구성

```
# Substrate 표준 팰릿
├── frame-system: 시스템 기본 기능
├── pallet-babe: BABE 합의 알고리즘
├── pallet-grandpa: GRANDPA 최종성
├── pallet-balances: 잔액 관리
├── pallet-transaction-payment: 트랜잭션 수수료
├── pallet-timestamp: 타임스탬프 관리

# 거버넌스 팰릿
├── pallet-democracy: 민주주의 거버넌스
├── pallet-collective: 위원회 관리
├── pallet-treasury: 재무 관리
```

```
├── pallet-sudo: 관리자 권한 (개발용)


# 스테이킹 팰릿
├── pallet-staking: 스테이킹 메커니즘
├── pallet-session: 세션 관리
├── pallet-im-online: 온라인 상태 확인
├── pallet-offences: 위반 행위 처리


# Phala 특화 팰릿
├── phala-pallets: Phala 핵심 팰릿
├── phat-offchain-rollup: 오프체인 롤업
└── pallet-mq-runtime-api: 메시지 큐 API
```

2.2 Runtime 설정

```rust
// Runtime 구성 예시 (추정)
#[derive(Clone, Eq, PartialEq)]
pub struct Runtime;


impl frame_system::Config for Runtime {
type BaseCallFilter = frame_support::traits::Everything;
type BlockWeights = BlockWeights;
type BlockLength = BlockLength;
```

```
type DbWeight = RocksDbWeight;

type RuntimeOrigin = RuntimeOrigin;

type RuntimeCall = RuntimeCall;

type Index = Index;

type BlockNumber = BlockNumber;

type Hash = Hash;

type Hashing = BlakeTwo256;

type AccountId = AccountId;

type Lookup = AccountIdLookup<AccountId, ()>;

type Header = generic::Header<BlockNumber, BlakeTwo256>;

type RuntimeEvent = RuntimeEvent;

type BlockHashCount = BlockHashCount;

type Version = Version;

type PalletInfo = PalletInfo;

type AccountData = pallet_balances::AccountData<Balance>;

type OnNewAccount = ();

type OnKilledAccount = ();

type SystemWeightInfo = ();

type SS58Prefix = SS58Prefix;

type OnSetCode = ();

type MaxConsumers = frame_support::traits::ConstU32<16>;

}
```

3. 합의 메커니즘 구현

3.1 BABE 설정

```
// BABE 팰릿 설정 (추정)

impl pallet_babe::Config for Runtime {

type EpochDuration = EpochDuration;

type ExpectedBlockTime = ExpectedBlockTime;

type EpochChangeTrigger = pallet_babe::ExternalTrigger;

type DisabledValidators = Session;

type KeyOwnerProof = <Self::KeyOwnerProofSystem as KeyOwnerProofSystem<(

KeyTypeId,

pallet_babe::AuthorityId,

)>>::Proof;

type KeyOwnerIdentification = <Self::KeyOwnerProofSystem as KeyOwnerProofSystem<

KeyTypeId,

pallet_babe::AuthorityId,

)>>::IdentificationTuple;

type KeyOwnerProofSystem = Historical;

type HandleEquivocation = pallet_babe::EquivocationHandler<

Self::KeyOwnerIdentification,

Offences,

ReportLongevity,
```

```
>;

type WeightInfo = ();

type MaxAuthorities = MaxAuthorities;

}


// BABE 에포크 설정

parameter_types! {

pub  const EpochDuration: u64 = EPOCH_DURATION_IN_SLOTS;

pub  const ExpectedBlockTime: Moment = MILLISECS_PER_BLOCK;

pub  const ReportLongevity: u64 =

BondingDuration::get() as u64 * SessionsPerEra::get() as u64 * EpochDuration::ge

}
```

3.2 GRANDPA 설정

```
// GRANDPA 팰릿 설정 (추정)

impl pallet_grandpa::Config for Runtime {

type RuntimeEvent = RuntimeEvent;

type KeyOwnerProof = <Self::KeyOwnerProofSystem as KeyOwnerProofSystem<(

KeyTypeId,

GrandpaId,

)>>::Proof;

type KeyOwnerIdentification = <Self::KeyOwnerProofSystem as KeyOwnerProofSystem<
```

```
KeyTypeId,

GrandpaId,

)>>::IdentificationTuple;

type KeyOwnerProofSystem = Historical;

type HandleEquivocation = pallet_grandpa::EquivocationHandler<

Self::KeyOwnerIdentification,

Offences,

ReportLongevity,

>;

type WeightInfo = ();

type MaxAuthorities = MaxAuthorities;

}
```

4. 경제 모델 구현

4.1 잔액 관리

```
// Balances 팰릿 설정 (추정)

impl pallet_balances::Config for Runtime {

type MaxLocks = MaxLocks;

type MaxReserves = MaxReserves;

type ReserveIdentifier = [u8; 8];

type Balance = Balance;
```

```
type RuntimeEvent = RuntimeEvent;

type DustRemoval = ();

type ExistentialDeposit = ExistentialDeposit;

type AccountStore = System;

type WeightInfo = pallet_balances::weights::SubstrateWeight<Runtime>;

}


// 경제 파라미터

parameter_types! {

pub  const ExistentialDeposit: Balance = 1 * CENTS;

pub  const MaxLocks: u32 = 50;

pub  const MaxReserves: u32 = 50;

}
```

4.2 트랜잭션 수수료

```
// Transaction Payment 설정 (추정)

impl pallet_transaction_payment::Config for Runtime {

type RuntimeEvent = RuntimeEvent;

type OnChargeTransaction = CurrencyAdapter<Balances, DealWithFees>;

type OperationalFeeMultiplier = OperationalFeeMultiplier;

type WeightToFee = WeightToFee;

type LengthToFee = ConstantMultiplier<Balance, TransactionByteFee>;
```

```
type FeeMultiplierUpdate = SlowAdjustingFeeUpdate<Self>;

}



// 수수료 계산 구조체

pub struct WeightToFee;

impl WeightToFeePolynomial for WeightToFee {

type Balance = Balance;

fn polynomial() -> WeightToFeeCoefficients<Self::Balance> {

// 선형 수수료 모델: weight * coefficient

let p = WEIGHT_PER_SECOND.ref_time();

let q = Balance::from(ExtrinsicBaseWeight::get().ref_time());

smallvec![WeightToFeeCoefficient {

degree: 1,

negative: false,

coeff_frac: Perbill::from_rational(p / q, 1),

coeff_integer: 0,

}]

}

}
```

5. 스테이킹 시스템

5.1 스테이킹 설정

```
// Staking 팰릿 설정 (추정)

impl pallet_staking::Config for Runtime {

type MaxNominatorRewardedPerValidator = MaxNominatorRewardedPerValidator;

type CurrencyToVote = U128CurrencyToVote;

type RewardRemainder = Treasury;

type RuntimeEvent = RuntimeEvent;

type Slash = Treasury;

type Reward = ();

type SessionsPerEra = SessionsPerEra;

type BondingDuration = BondingDuration;

type SlashDeferDuration = SlashDeferDuration;

type AdminOrigin = EitherOfDiverse<

EnsureRoot<AccountId>,

pallet_collective::EnsureProportionAtLeast<AccountId, CouncilCollective, 3, 4>,

>;

type SessionInterface = Self;

type EraPayout = pallet_staking::ConvertCurve<RewardCurve>;

type NextNewSession = Session;

type MaxNominations = MaxNominations;

type TargetList = pallet_staking::UseValidatorsMap<Self>;

type MaxUnlockingChunks = MaxUnlockingChunks;

type HistoryDepth = HistoryDepth;
```

```
type OnStakerSlash = NominationPools;

type WeightInfo = pallet_staking::weights::SubstrateWeight<Runtime>;

}


// 스테이킹 파라미터

parameter_types! {

pub  const SessionsPerEra: sp_staking::SessionIndex = 6;

pub  const BondingDuration: sp_staking::EraIndex = 24 * 28;

pub  const SlashDeferDuration: sp_staking::EraIndex = 24 * 7;

pub  const MaxNominations: u32 = 16;

pub  const MaxNominatorRewardedPerValidator: u32 = 256;

pub  const MaxUnlockingChunks: u32 = 32;

pub  const HistoryDepth: u32 = 84;

}
```

5.2 보상 곡선

```
// 스테이킹 보상 곡선 (추정)

pallet_staking_reward_curve::build! {

const REWARD_CURVE: PiecewiseLinear<'static> = curve!(

min_inflation: 0_025_000,

max_inflation: 0_100_000,

ideal_stake: 0_500_000,
```

```
falloff: 0_050_000,
max_piece_count: 40,
test_precision: 0_005_000,
);
}

parameter_types! {
pub  const RewardCurve: &'static PiecewiseLinear<'static> = &REWARD_CURVE;
}
```

6. 거버넌스 시스템

6.1 민주주의 거버넌스

```
// Democracy 팰릿 설정 (추정)
impl pallet_democracy::Config for Runtime {
type RuntimeEvent = RuntimeEvent;
type Currency = Balances;
type EnactmentPeriod = EnactmentPeriod;
type LaunchPeriod = LaunchPeriod;
type VotingPeriod = VotingPeriod;
type VoteLockingPeriod = EnactmentPeriod;
type MinimumDeposit = MinimumDeposit;
```

```
type ExternalOrigin = EitherOfDiverse<
pallet_collective::EnsureProportionAtLeast<AccountId, CouncilCollective, 1, 2>,
frame_system::EnsureRoot<AccountId>,
>;
type ExternalMajorityOrigin = EitherOfDiverse<
pallet_collective::EnsureProportionAtLeast<AccountId, CouncilCollective, 3, 4>,
frame_system::EnsureRoot<AccountId>,
>;
type ExternalDefaultOrigin = EitherOfDiverse<
pallet_collective::EnsureProportionAtLeast<AccountId, CouncilCollective, 1, 1>,
frame_system::EnsureRoot<AccountId>,
>;
type FastTrackOrigin = EitherOfDiverse<
pallet_collective::EnsureProportionAtLeast<AccountId, TechnicalCollective, 2, 3>
frame_system::EnsureRoot<AccountId>,
>;
type InstantOrigin = EitherOfDiverse<
pallet_collective::EnsureProportionAtLeast<AccountId, TechnicalCollective, 1, 1>
frame_system::EnsureRoot<AccountId>,
>;
type InstantAllowed = InstantAllowed;
type FastTrackVotingPeriod = FastTrackVotingPeriod;
type CancellationOrigin = EitherOfDiverse<
```

```
EnsureRoot<AccountId>,
pallet_collective::EnsureProportionAtLeast<AccountId, CouncilCollective, 2, 3>,
>;
type BlacklistOrigin = EnsureRoot<AccountId>;
type CancelProposalOrigin = EitherOfDiverse<
EnsureRoot<AccountId>,
pallet_collective::EnsureProportionAtLeast<AccountId, TechnicalCollective, 1, 1>
>;
type VetoOrigin = pallet_collective::EnsureMember<AccountId, TechnicalCollective
type CooloffPeriod = CooloffPeriod;
type PreimageByteDeposit = PreimageByteDeposit;
type OperationalPreimageOrigin = pallet_collective::EnsureMember<AccountId, Coun
type Slash = Treasury;
type Scheduler = Scheduler;
type PalletsOrigin = OriginCaller;
type MaxVotes = MaxVotes;
type WeightInfo = pallet_democracy::weights::SubstrateWeight<Runtime>;
type MaxProposals = MaxProposals;
}
```

6.2 위원회 시스템

```
// Council 설정 (추정)
```

```
type CouncilCollective = pallet_collective::Instance1;

impl pallet_collective::Config<CouncilCollective> for Runtime {

type RuntimeOrigin = RuntimeOrigin;

type Proposal = RuntimeCall;

type RuntimeEvent = RuntimeEvent;

type MotionDuration = CouncilMotionDuration;

type MaxProposals = CouncilMaxProposals;

type MaxMembers = CouncilMaxMembers;

type DefaultVote = pallet_collective::PrimeDefaultVote;

type WeightInfo = pallet_collective::weights::SubstrateWeight<Runtime>;

}



// Technical Committee 설정

type TechnicalCollective = pallet_collective::Instance2;

impl pallet_collective::Config<TechnicalCollective> for Runtime {

type RuntimeOrigin = RuntimeOrigin;

type Proposal = RuntimeCall;

type RuntimeEvent = RuntimeEvent;

type MotionDuration = TechnicalMotionDuration;

type MaxProposals = TechnicalMaxProposals;

type MaxMembers = TechnicalMaxMembers;

type DefaultVote = pallet_collective::PrimeDefaultVote;

type WeightInfo = pallet_collective::weights::SubstrateWeight<Runtime>;
```

```
}
```

7. Phala 특화 팰릿 통합

7.1 Phala Pallets 설정

```
// Phala 팰릿 설정 (추정)
impl phala_pallets::Config for Runtime {
type RuntimeEvent = RuntimeEvent;
type Currency = Balances;
// TEE 워커 관련 설정
type WorkerLifetime = WorkerLifetime;
type MinWorkerDeposit = MinWorkerDeposit;
// 클러스터 관련 설정
type MaxClustersPerWorker = MaxClustersPerWorker;
type ClusterDeposit = ClusterDeposit;
// 컨트랙트 관련 설정
type MaxContractsPerCluster = MaxContractsPerCluster;
type ContractDeposit = ContractDeposit;
// 거버넌스 권한
type GovernanceOrigin = EitherOfDiverse<
EnsureRoot<AccountId>,
pallet_collective::EnsureProportionAtLeast<AccountId, CouncilCollective, 1, 2>,
```

```
>;
type WeightInfo = ();
}

parameter_types! {
pub const WorkerLifetime: BlockNumber = 7 * DAYS;
pub const MinWorkerDeposit: Balance = 100 * DOLLARS;
pub const MaxClustersPerWorker: u32 = 10;
pub const ClusterDeposit: Balance = 10 * DOLLARS;
pub const MaxContractsPerCluster: u32 = 100;
pub const ContractDeposit: Balance = 1 * DOLLARS;
}
```

7.2 메시지 큐 API

```
// MQ Runtime API 구현 (추정)
impl pallet_mq_runtime_api::MqApi<Block, Hash> for Runtime {
fn mq_egress_messages() -> Vec<EgressMessage> {
PhalaRegistry::mq_egress_messages()
}
fn mq_sequence() -> u64 {
PhalaRegistry::mq_sequence()
}
```

```
fn mq_process_messages(messages: Vec<IngressMessage>) -> Vec<ProcessedMessage>
PhalaRegistry::mq_process_messages(messages)
}
}
```

8. Runtime API 구현

8.1 표준 Runtime API

```
// Runtime API 구현 (추정)
impl_runtime_apis! {
impl sp_api::Core<Block> for Runtime {
fn version() -> RuntimeVersion {
VERSION
}
fn execute_block(block: Block) {
Executive::execute_block(block);
}
fn initialize_block(header: &<Block as BlockT>::Header) {
Executive::initialize_block(header)
}
}
impl sp_api::Metadata<Block> for Runtime {
```

```
fn metadata() -> OpaqueMetadata {
OpaqueMetadata::new(Runtime::metadata().into())
}
}
impl sp_block_builder::BlockBuilder<Block> for Runtime {
fn apply_extrinsic(extrinsic: <Block as BlockT>::Extrinsic) -> ApplyExtrinsicRe
Executive::apply_extrinsic(extrinsic)
}
fn finalize_block() -> <Block as BlockT>::Header {
Executive::finalize_block()
}
fn inherent_extrinsics(data: sp_inherents::InherentData) -> Vec<<Block as Block
data.create_extrinsics()
}
fn check_inherents(
block: Block,
data: sp_inherents::InherentData,
) -> sp_inherents::CheckInherentsResult {
data.check_extrinsics(&block)
}
}
}
```

8.2 Phala 특화 Runtime API

```
// Phala Runtime API 구현 (추정)
impl_runtime_apis! {
impl phala_pallets::PhalaApi<Block> for Runtime {
fn  get_worker_info(worker_id: WorkerId) -> Option<WorkerInfo> {
PhalaRegistry::worker_info(&worker_id)
}
fn  get_cluster_info(cluster_id: ClusterId) -> Option<ClusterInfo> {
PhalaRegistry::cluster_info(&cluster_id)
}
fn  get_contract_info(contract_id: ContractId) -> Option<ContractInfo> {
PhalaRegistry::contract_info(&contract_id)
}
fn  pink_query(
contract_id: ContractId,
origin: AccountId,
input: Vec<u8>,
) -> Result<Vec<u8>, String> {
PhalaRegistry::pink_query(contract_id, origin, input)
}
}
}
```

9. Executive 및 상태 전이

9.1 Executive 설정

```
// Executive 구성 (추정)
pub type Executive = frame_executive::Executive<
Runtime,
Block,
frame_system::ChainContext<Runtime>,
Runtime,
AllPalletsWithSystem,
Migrations,
>;

// 마이그레이션 정의
pub struct Migrations;
impl OnRuntimeUpgrade for Migrations {
fn on_runtime_upgrade() -> Weight {
// 런타임 업그레이드 시 실행될 마이그레이션 로직
let mut weight = Weight::zero();
// 각 팰릿별 마이그레이션 실행
weight = weight.saturating_add(migrate_phala_pallets());
weight = weight.saturating_add(migrate_staking_pallet());
```

```
weight
}
}
```

9.2 블록 실행 과정

```
// 블록 실행 로직 (추정)
impl Executive {
fn execute_block_impl(block: Block) -> Result<(), DispatchError> {
// 1. 블록 헤더 초기화
Self::initialize_block(&block.header());
// 2. Inherent 트랜잭션 실행
for extrinsic in block.extrinsics().iter().filter(|ext| ext.is_inherent()) {
Self::apply_extrinsic(extrinsic.clone())?;
}
// 3. 일반 트랜잭션 실행
for extrinsic in block.extrinsics().iter().filter(|ext| !ext.is_inherent()) {
let result = Self::apply_extrinsic(extrinsic.clone());
// 트랜잭션 실패 시에도 블록 실행 계속
if let Err(e) = result {
log::warn!("Transaction failed: {:?}", e);
}
}
```

```
// 4. 블록 마무리

let  header = Self::finalize_block();

// 5. 헤더 검증

if  header.hash() != block.header().hash() {

return Err(DispatchError::Other("Header hash mismatch"));

}

Ok(())

}

}
```

이 문서는 Phala Network Runtime의 구현을 상세히 다룹니다. 다음 문서에서는 Phala 특화 팰릿들의 구현을 분석합니다.